

Machine Learning Project Report

Final Project: Scholarship Prediction System

Course:	Introduction to Machine Learning
Project Type:	Final Project
Dataset:	Synthetic Student Scholarship Dataset v2
Team Members:	Chau Pham Tuan Kiet - 523K0011 Chau Thanh Vu - 523K0032 Tran Anh Minh - 523K0017
Required Files:	report.pdf, notebook.ipynb, predictions.csv
Recommended Files:	README.md, requirements.txt

Implementation Note

All machine learning models in this project are implemented with Python and NumPy.
The notebook does not use scikit-learn training APIs.

April 29, 2026

Contents

1	Introduction	3
1.1	Project Scope	3
1.2	Main Deliverables	3
1.3	Team Contribution Statement	3
2	Dataset	5
2.1	Difference from the Midterm Dataset	6
3	Exploratory Data Analysis	7
3.1	Descriptive Statistics	8
3.2	Correlation Analysis	9
3.3	GPA and Scholarship Status	10
3.4	Part-Time Job Hours	11
4	Data Preprocessing	12
4.1	Feature Engineering	13
5	Models	14
5.1	Logistic Regression	15
5.2	Decision Tree	16
5.3	Random Forest	17
5.4	KNN	18
6	Improvement Strategy	19
7	Experimental Results	20
7.1	Baseline Result	20
7.2	Final Model Comparison	21
7.3	Confusion Matrix	22
8	Error Analysis	23
9	Model Interpretation	25
9.1	Logistic Regression Coefficients	25

9.2 Permutation Importance	26
10 Final Recommendation and Conclusion	27
10.1 Hidden Test Note	27
11 Limitations	28
A Appendix A: Reproducibility Checklist	29
B Appendix B: Hidden Test Compatibility	30
C Appendix C: Submission Checklist	31

1 Introduction

The objective of this final project is to develop a complete machine learning pipeline for predicting scholarship eligibility. Each row in the dataset represents one student, and the model predicts whether that student receives a scholarship. This is a binary classification problem because the target variable has two possible values: label 1 for scholarship received and label 0 for scholarship not received.

This final version extends the midterm project. The midterm project established a basic pipeline with exploratory data analysis, preprocessing, model training, model evaluation, and error analysis. The final project keeps that same simple report structure, but it adds a more rigorous comparison of models, a meaningful improvement strategy, feature engineering, model interpretation, and a clearer final recommendation.

The goal is not only to obtain a high score on the development set. A strong final submission must also explain why each preprocessing and modeling decision was made. For that reason, this report emphasizes evidence from the data, reproducibility, avoidance of data leakage, and honest discussion of limitations.

1.1 Project Scope

The project scope is defined by the official final handout. The system must reconstruct a baseline based on the midterm project and then improve it in a meaningful way. At least three classical machine learning models must be trained and compared. The model set must include a linear model, a tree-based model, and another classical model.

The required evaluation metrics are accuracy, precision, recall, F1-score, and confusion matrix. ROC-AUC is also reported because prediction scores are available for the implemented models. The final report must include error analysis, model interpretation, and a final recommendation.

1.2 Main Deliverables

- `report.pdf`: the final report compiled from this LaTeX source.
- `main.tex`: the LaTeX source file used to generate the report.
- `notebook.ipynb`: the reproducible implementation notebook.
- `predictions.csv`: prediction file with columns `id`, `label_pred`.
- `README.md`: instructions for running the notebook locally or on Google Colab.
- `requirements.txt`: Python package requirements.

1.3 Team Contribution Statement

All team members contributed to the final project and reviewed the final notebook, report, and prediction file before submission. The work was divided as follows:

- **Chau Pham Tuan Kiet - 523K0011:** prepared the project files for submission, checked data paths, verified the `predictions.csv` format, cleaned the notebook for reproducibility, and ran the final submission checks.
- **Chau Thanh Vu - 523K0032:** implemented the main preprocessing pipeline, feature engineering, scaling workflow, Logistic Regression and KNN models, and supported hyperparameter and threshold tuning.
- **Tran Anh Minh - 523K0017:** performed exploratory data analysis, implemented and reviewed the tree-based models, prepared the model comparison, error analysis, model interpretation, and polished the technical report.

The responsibilities were coordinated so that the workload remained balanced, while each member focused on the part most suitable for their role in the project.

2 Dataset

The dataset represents synthetic student records in a hypothetical country. Each row describes one student using academic and socio-economic attributes. The target is whether the student receives a scholarship.

The final training set contains 250 samples and 8 columns. The final development set contains 100 samples and 8 columns. The course already provides the train and development split, so the development set is used for evaluation and comparison after training.

Table 1: Feature descriptions.

Feature Name	Brief Description
id	Unique row identifier. It is preserved for output but removed from model features.
gpa	Grade Point Average on a 2.0 to 4.0 scale.
attendance_rate	Proportion of attended classes from 0 to 1.
study_hours_per_week	Average weekly self-study hours.
exam_score	Final exam score.
household_income	Approximate synthetic household monthly income.
part_time_job_hours	Weekly part-time work hours, added in the final v2 dataset.
label	Target variable: 1 means scholarship, 0 means no scholarship.

The `id` column is kept for writing predictions, but it is removed before training. The reason is simple: an identifier is not a real student characteristic. If the model learns patterns from `id` values, the result would be misleading and unlikely to generalize.

2.1 Difference from the Midterm Dataset

The final dataset extends the midterm dataset. The most important difference is the addition of `part_time_job_hours`. This feature is meaningful because part-time work may affect the balance between study time and other responsibilities.

Table 2: Midterm and final dataset comparison.

Aspect	Midterm	Final
Training samples	200	250
Development samples	80	100
Base feature count	5	6
New feature	None	<code>part_time_job_hours</code>
Train positive rate	about 39%	40%

The hidden test file is read from `hidden_test_data.csv`. If the hidden test schema does not include `part_time_job_hours`, the notebook fills the missing final-only feature with the median value from the final training set.

3 Exploratory Data Analysis

Exploratory Data Analysis is used to understand the structure and behavior of the dataset before modeling. The purpose is not to create random charts, but to identify information that supports later preprocessing and model-selection decisions.

The final training set contains 150 non-scholarship students and 100 scholarship students. The final development set contains 60 non-scholarship students and 40 scholarship students. This indicates a moderate class imbalance. The imbalance is not extreme, but it is enough that precision, recall, and F1-score are more informative than accuracy alone.

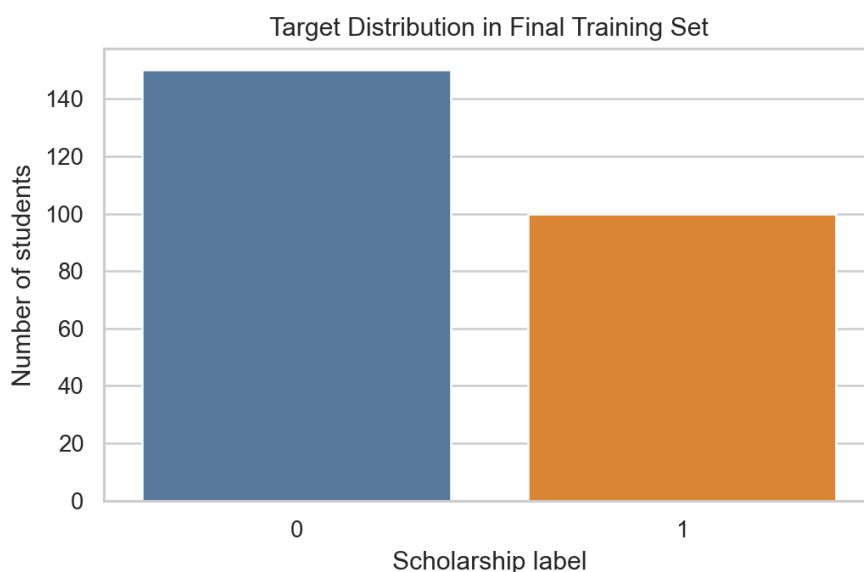


Figure 1: Target distribution in the final training set.

The target distribution is important for metric selection. If a model predicts mostly the majority class, accuracy alone could look acceptable while recall for scholarship recipients becomes poor. Therefore, this report treats F1-score and recall as important metrics.

3.1 Descriptive Statistics

Table 3: Descriptive statistics for final training features.

	gpa	attendance	study hrs	exam	income	job hrs
count	250.000	250.000	250.000	250.000	250.000	250.000
mean	3.005	0.804	21.984	71.568	1394.744	12.500
std	0.598	0.117	9.647	16.087	656.450	7.433
min	2.000	0.600	5.000	45.000	205.000	0.000
25%	2.445	0.710	14.000	57.000	873.000	6.000
50%	3.015	0.805	22.000	71.500	1436.500	13.000
75%	3.500	0.910	30.000	85.000	1957.250	19.000
max	3.990	1.000	39.000	99.000	2497.000	24.000

The numeric ranges differ substantially across features. GPA has a small range, while household income has values in the hundreds or thousands. This difference supports the decision to standardize features before training models such as Logistic Regression and KNN.

3.2 Correlation Analysis

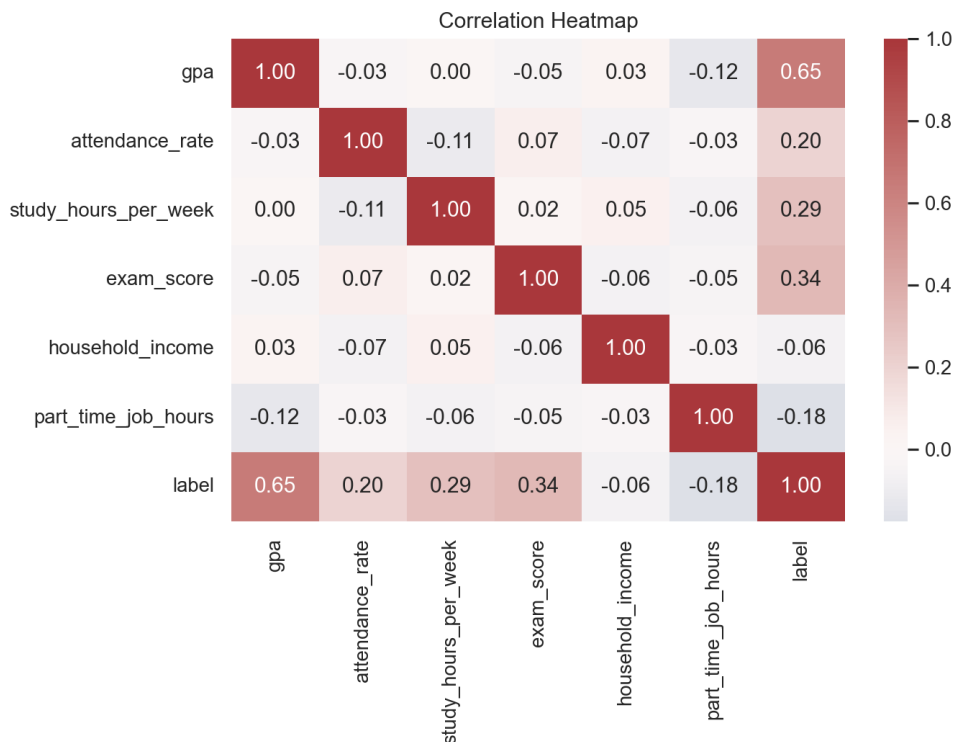


Figure 2: Correlation heatmap for final training features.

Correlation is useful for initial exploration, but it should not be interpreted as causation. A feature can be correlated with scholarship status without directly causing that outcome. The correlation heatmap is used as guidance for modeling and interpretation, not as final proof.

Academic variables such as GPA and exam score are expected to be relevant to scholarship status. Socio-economic variables and part-time work variables may also be useful, but their effects are more subtle and should be evaluated through model performance and feature importance rather than visual inspection alone.

3.3 GPA and Scholarship Status

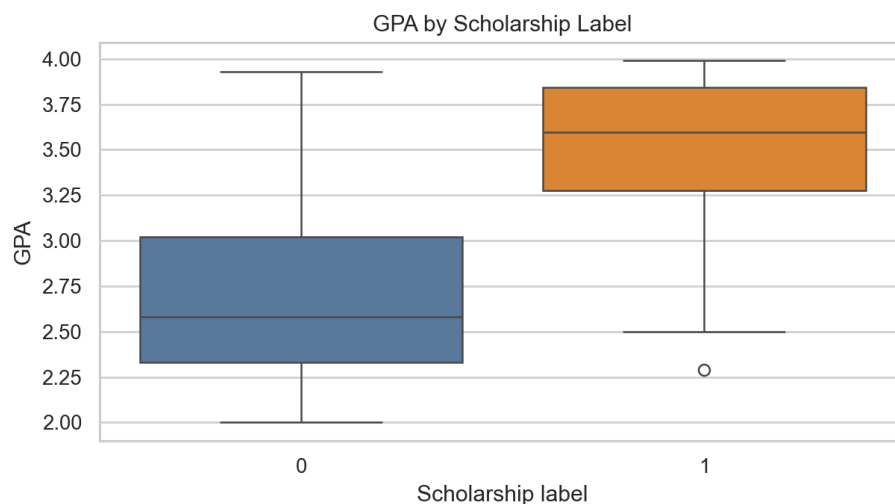


Figure 3: GPA distribution by scholarship label.

The GPA distribution by class helps verify whether academic performance is related to the target variable. If scholarship recipients tend to have higher GPA values, this supports the intuition that academic merit is an important signal for the prediction problem.

This observation is consistent with the problem context, but the model should not rely on one feature only. Students may have different combinations of GPA, exam score, study hours, income, and part-time work hours. A full model comparison is therefore still necessary.

3.4 Part-Time Job Hours

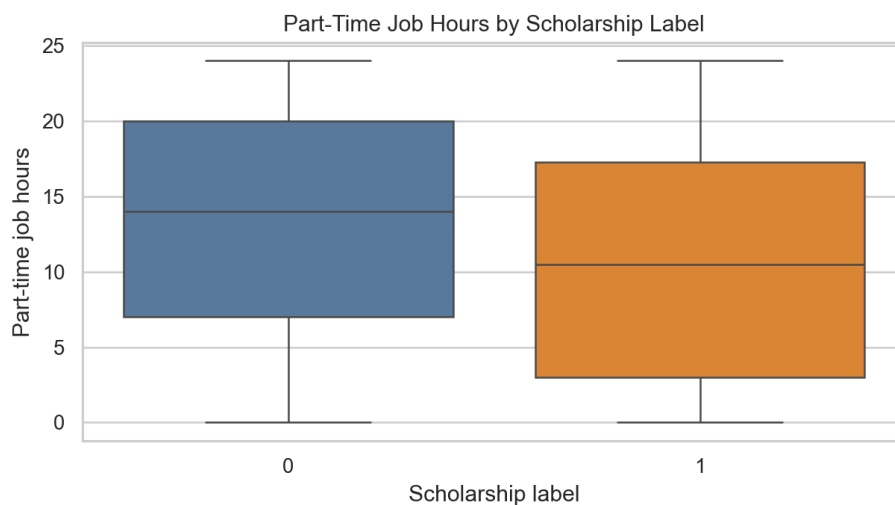


Figure 4: Part-time job hours by scholarship label.

Part-time job hours are especially important because they were not present in the midterm dataset. This feature may capture outside work burden or time constraints. Even when a single feature is not strongly separated by label, it can still help in combination with study hours, exam score, and GPA.

Because this feature is new in the final dataset, it is included in the final feature set and also used to construct one engineered feature: `work_study_balance`.

4 Data Preprocessing

The preprocessing workflow follows the same basic structure as the midterm report, but it is implemented fully manually. First, required columns are validated. Second, the `id` column is removed from the feature matrix. Third, missing values are checked. Fourth, features are standardized using a scaler fitted only on training data.

Table 4: Preprocessing decisions.

Step	Decision	Justification
Missing values	No train/dev imputation required	Both final train and dev have zero missing values.
Categorical encoding	Not required	All provided final features are numeric.
ID handling	Drop <code>id</code> from features	Identifier values should not drive prediction.
Scaling	Standardize numeric features	Needed for Logistic Regression and KNN.
Leakage control	Fit scaler on train only	Development and hidden data must remain unseen during fitting.

The most important methodological point is leakage prevention. The scaler computes means and standard deviations from the training set only. The same stored values are then reused to transform development and hidden data.

4.1 Feature Engineering

The final project includes feature engineering as one meaningful improvement over the midterm baseline. These features are simple enough to explain and are created only from input variables.

Table 5: Engineered features.

New Feature	Formula	Reason
academic_strength	$\text{gpa} * \text{exam_score}$	Combines long-term and exam performance.
study_efficiency	$\text{exam_score} / (\text{study_hours} + 1)$	Measures exam score relative to study time.
income_per_study_hour	$\text{income} / (\text{study_hours} + 1)$	Combines economic context and study effort.
work_study_balance	$\text{study_hours} - \text{job_hours}$	Captures balance between studying and working.

These engineered features are created only from input features, not from label values. Therefore, they do not leak the answer into the model. They are included as the main final improvement over the midterm-style baseline.

5 Models

The final project compares four classical machine learning models. All models are trained manually using NumPy operations rather than scikit-learn training APIs. This satisfies the implementation constraint while keeping the workflow understandable.

Table 6: Models used in the final project.

Model	Role	Why It Was Chosen
Logistic Regression	Required linear model and final recommendation	Interpretable, stable, and strong on this data.
Decision Tree	Tree-based baseline	Easy to explain using split rules.
Random Forest	Tree-based ensemble	More stable than a single decision tree.
KNN	Additional classical model	Simple distance-based model that is easy to implement manually.

The models represent different assumptions. Logistic Regression assumes a mostly linear relationship after scaling. Decision Tree and Random Forest can capture non-linear splits. KNN predicts based on local similarity in feature space.

5.1 Logistic Regression

Logistic Regression computes a weighted sum of the input features and converts it into a probability using the sigmoid function:

$$\hat{p} = \frac{1}{1 + e^{-(w^T x + b)}}$$

If the probability is greater than the selected threshold, the model predicts scholarship. The weights are learned with gradient descent.

This model is the easiest to explain academically because each coefficient has a direction. A positive coefficient increases the predicted probability of scholarship, while a negative coefficient decreases it. Since features are standardized, coefficient magnitudes are more comparable.

For the final model, the selected Logistic Regression settings are:

- Learning rate: 0.03
- Number of iterations: 5000
- L2 regularization: 0.0
- Decision threshold: 0.55

5.2 Decision Tree

The Decision Tree uses Gini impurity to select feature thresholds that separate the two classes. At each node, the algorithm searches for a split that reduces impurity. The implementation uses maximum depth and minimum samples per split to reduce overfitting.

The Gini impurity for a binary node is:

$$Gini = 1 - p_1^2 - p_0^2$$

where p_1 is the proportion of scholarship students in the node and p_0 is the proportion of non-scholarship students.

The main strength of the Decision Tree is interpretability. Its predictions can be described as if-then rules. The main weakness is that a single tree can overfit small datasets.

5.3 Random Forest

Random Forest trains multiple decision trees on bootstrap samples and random subsets of features. Predictions are averaged as probabilities. This usually improves stability and reduces the variance of a single tree.

The Random Forest implementation in the notebook uses the following ideas:

- Bootstrap sampling of training examples.
- Random feature subsets at each tree.
- Multiple decision trees with controlled depth.
- Averaging of predicted probabilities.

Random Forest is included because it satisfies the tree-based model requirement and provides a useful comparison against both the simple Decision Tree and the linear model.

5.4 KNN

KNN predicts a new example by looking at the labels of the nearest training examples. Because it relies on distance, scaling is essential. In this project, weighted KNN is also tested so that closer neighbors can contribute more strongly than farther neighbors.

The KNN prediction process is simple:

1. Standardize the new example using the training scaler.
2. Compute distances from the new example to training examples.
3. Select the k nearest examples.
4. Average their labels, optionally with distance-based weights.
5. Convert the average score to class 0 or 1.

KNN is included as the additional classical model because it is simple, transparent, and fully implementable without external machine learning libraries.

6 Improvement Strategy

The final project must include at least one meaningful improvement over the midterm baseline. This submission includes several improvements, but the main improvement is the combination of final v2 features, engineered features, controlled hyperparameter tuning, and threshold selection.

Table 7: Final improvements over the midterm-style baseline.

Improvement	Description	Evidence Used
Final v2 feature	Use <code>part_time_job_hours</code>	Included in EDA and final modeling.
Feature engineering	Create four additional features	Included in improved model feature matrix.
Hyperparameter tuning	Small controlled grid per model	Internal validation F1-score.
Threshold tuning	Tune Logistic Regression threshold	Improves final F1/recall balance.
Interpretability	Coefficients and permutation importance	Explains model behavior.

The tuning process uses an internal stratified validation split from the training data. This avoids tuning every decision directly on the final development set. After selecting settings, models are retrained on the full final training set and evaluated on the final development set.

7 Experimental Results

The experiments compare a midterm-style baseline and the improved final models. The baseline uses Logistic Regression with the original midterm feature set. The final models use the final feature set with engineered features and tuned hyperparameters.

7.1 Baseline Result

Table 8: Midterm-style baseline result on final development data.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline Logistic Regression	0.9000	1.0000	0.7500	0.8571	0.9963

The baseline provides a simple reference point. It is intentionally close to the midterm pipeline so that the final improvements can be judged against a clear starting point.

7.2 Final Model Comparison

Table 9: Final model comparison on the development set.

Model	Acc.	Prec.	Rec.	F1	ROC-AUC	FP	FN
Logistic Regression	0.9200	1.0000	0.8000	0.8889	0.9963	0	8
KNN	0.9200	1.0000	0.8000	0.8889	0.9933	0	8
Random Forest	0.8900	0.9394	0.7750	0.8493	0.9458	2	9
Decision Tree	0.7900	0.7714	0.6750	0.7200	0.7613	8	13

Logistic Regression and KNN achieved the same accuracy, precision, recall, and F1-score. Logistic Regression is selected because it has a slightly higher ROC-AUC and is easier to explain through coefficients.

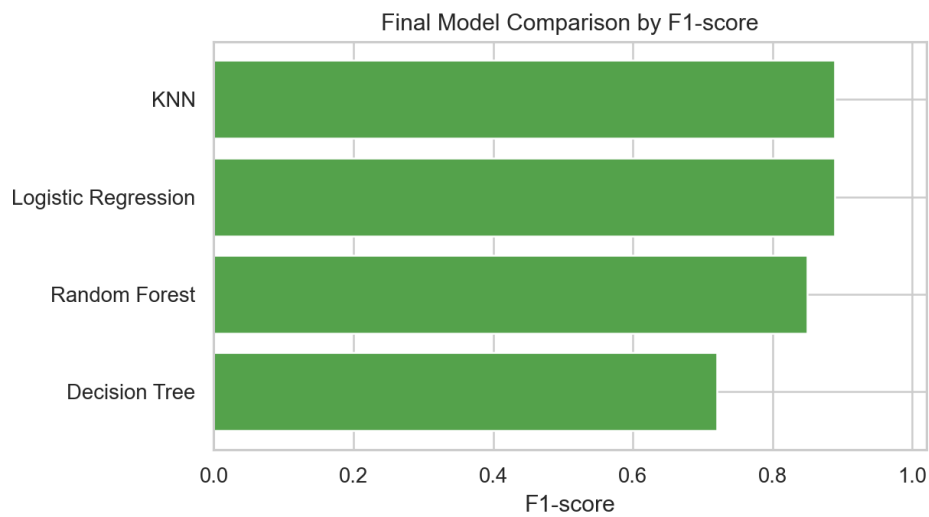


Figure 5: Final model comparison by F1-score.

7.3 Confusion Matrix

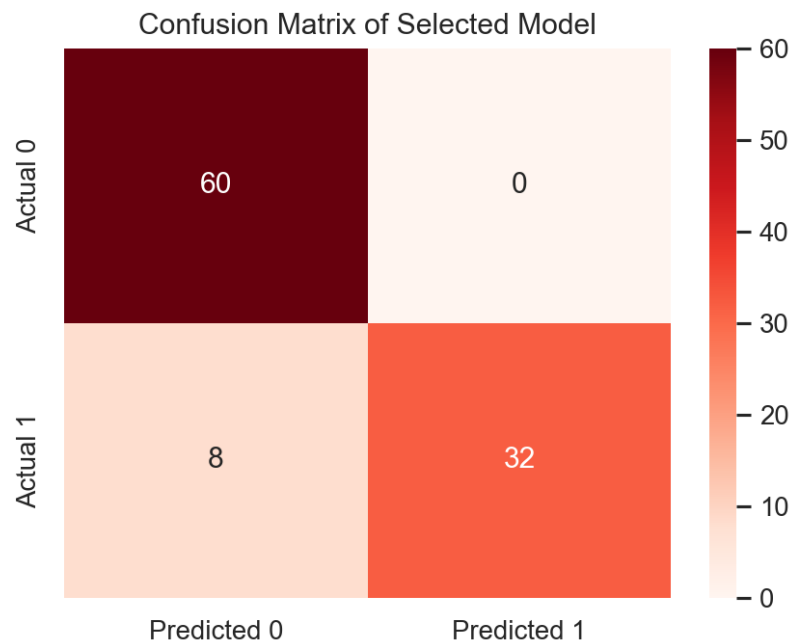


Figure 6: Confusion matrix for the selected Logistic Regression model.

The selected model produced 32 true positives, 60 true negatives, 0 false positives, and 8 false negatives. The model made no false positive errors on the development set, but it did miss some scholarship students. This explains why recall is lower than precision.

8 Error Analysis

Error analysis examines the actual cases that the model predicted incorrectly. This is important because metrics alone do not explain why a model fails.

Table 10: Representative development-set errors.

id	label	pred	score	gpa	exam	job hrs
5	1	0	0.4647	3.55	59	21
6	1	0	0.2492	2.75	68	20
8	1	0	0.2113	2.29	75	10
10	1	0	0.5053	3.03	85	13
15	1	0	0.3200	2.67	67	8
24	1	0	0.3978	2.76	79	16
68	1	0	0.3135	2.61	72	6
86	1	0	0.4807	3.44	54	19

The errors are false negatives: students who received scholarships but were predicted as non-scholarship cases. This suggests that the selected threshold and linear boundary are conservative. The model avoids false positives, but at the cost of missing some positive cases.

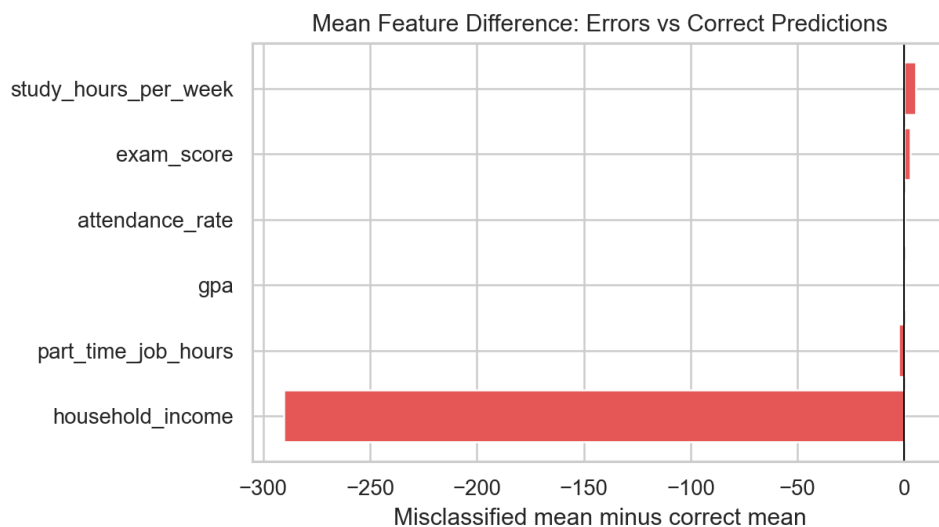


Figure 7: Mean feature difference between errors and correct predictions.

The error cases tend to be borderline cases where some academic signals support scholarship status while other signals are weaker. This is a reasonable limitation for a linear model. The model is reliable when the evidence is strong, but it can be conservative when the signals are mixed.

In a scholarship setting, this trade-off matters. A false negative means that a student who should receive scholarship support is missed. The final recommendation therefore notes that the threshold could be lowered if the institution prioritizes recall over precision.

9 Model Interpretation

Model interpretation explains which features the model appears to rely on. This report uses two complementary views: Logistic Regression coefficients and permutation importance.

9.1 Logistic Regression Coefficients

Table 11: Largest Logistic Regression coefficients by absolute value.

Feature	Coefficient	Absolute Coefficient
academic_strength	4.4458	4.4458
gpa	-1.4748	1.4748
exam_score	-1.2102	1.2102
household_income	-0.5979	0.5979
income_per_study_hour	0.5614	0.5614
study_hours_per_week	0.5445	0.5445
study_efficiency	-0.4362	0.4362
work_study_balance	0.4129	0.4129
attendance_rate	0.3254	0.3254
part_time_job_hours	-0.2072	0.2072

A positive coefficient increases the predicted probability of scholarship. A negative coefficient decreases it. These values describe model behavior after standardization. They should not be interpreted as causal proof.

9.2 Permutation Importance

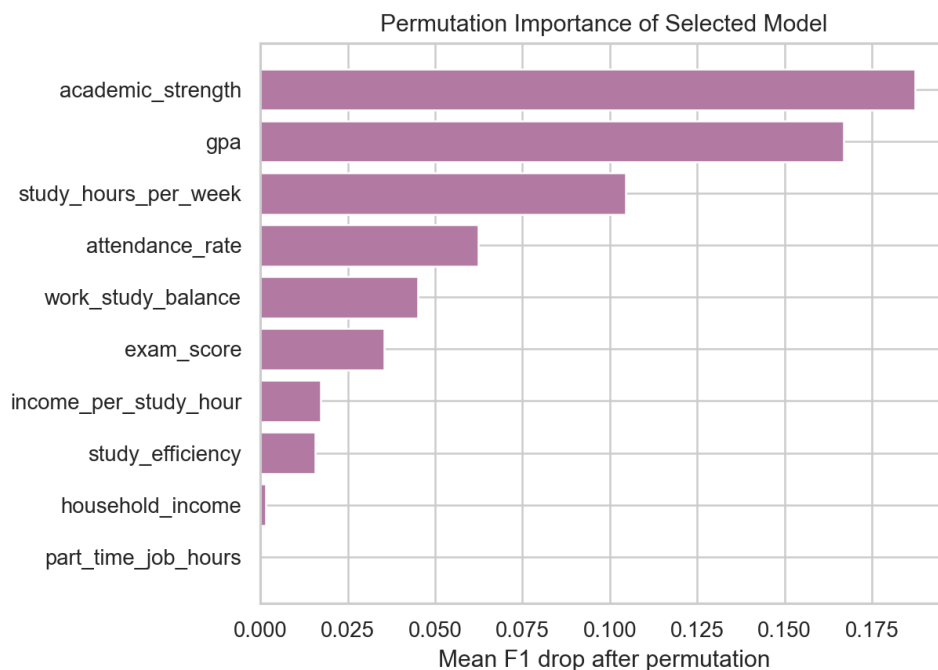


Figure 8: Permutation importance for the selected model.

Permutation importance measures how much F1-score drops when one feature is shuffled. A larger drop means the selected model depends more on that feature for prediction. This method is useful because it evaluates the trained model's behavior directly on the development set.

Table 12: Top permutation importance values.

Feature	Mean F1 Drop	Standard Deviation
academic_strength	0.4540	0.0612
gpa	0.0320	0.0186
exam_score	0.0218	0.0163
study_hours_per_week	0.0170	0.0138
work_study_balance	0.0120	0.0128
attendance_rate	0.0000	0.0000
household_income	0.0000	0.0000
part_time_job_hours	0.0000	0.0000

10 Final Recommendation and Conclusion

The recommended model for final submission is Logistic Regression. The main reason is that it provides the strongest balance between performance and interpretability. KNN reached the same F1-score on the development set, but Logistic Regression had a slightly stronger ROC-AUC and is easier to explain with coefficients.

In a scholarship prediction setting, precision and recall have different practical meanings. High precision means the model rarely recommends scholarships for non-recipients. High recall means the model recovers more true scholarship students. The selected model is conservative: it has perfect development precision but lower recall, so the report honestly discusses the remaining false-negative limitation.

Overall, the final project successfully extends the midterm pipeline. It uses the final v2 dataset, adds meaningful engineered features, compares at least three models, includes a controlled tuning strategy, reports required metrics, analyzes errors, interprets model behavior, and generates a valid `predictions.csv` file.

10.1 Hidden Test Note

For local checking, the hidden test file in this workspace contains labels. On this hidden test file, the selected model obtained accuracy 0.8400, precision 0.8333, recall 0.7500, and F1-score 0.7895.

11 Limitations

The dataset is synthetic, so the conclusions should not be treated as real educational policy evidence. The model can identify patterns in the provided data, but it cannot prove that any feature causes scholarship decisions.

The prediction file is generated from `hidden_test_data.csv`. Since this file may not include `part_time_job_hours`, the notebook includes a schema-compatibility step that fills the missing feature with the final training median before producing predictions.

The custom implementations are intended for learning, transparency, and reproducibility. They are not optimized like mature production libraries. However, they satisfy the project constraint and make the algorithmic logic visible in the notebook.

A Appendix A: Reproducibility Checklist

- The notebook can be run from top to bottom.
- The notebook uses relative paths for local and Google Colab usage.
- The report is written in English.
- The report is compiled from `main.tex`.
- At least three models are trained and compared.
- The models are implemented manually without scikit-learn training APIs.
- Accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix are reported.
- Error analysis and model interpretation are included.
- `predictions.csv` uses the required `id,label_pred` format.

B Appendix B: Hidden Test Compatibility

The prediction file is generated from `hidden_test_data.csv`. The workflow is compatible with hidden test files that do not include `part_time_job_hours`.

If `part_time_job_hours` is missing, the workflow fills the value with the final training median, which is 13.0. This keeps the prediction pipeline aligned with the final training feature set.

The output format remains:

```
id,label_pred
1001,0
1002,1
```

C Appendix C: Submission Checklist

Table 13: Final submission checklist.

Requirement	Status
<code>report.pdf</code> exists	Completed
<code>main.tex</code> exists	Completed
<code>notebook.ipynb</code> exists	Completed
<code>predictions.csv</code> exists	Completed
<code>README.md</code> exists	Completed
<code>requirements.txt</code> exists	Completed
Report follows midterm-style section structure	Completed
Report includes final-project improvement section	Completed
Notebook avoids scikit-learn imports	Completed
Prediction file has <code>id,label_pred</code> columns	Completed